

Étape 3

Machine learning

Prédire le sens de la prochaine bougie, mesurer honnêtement ce que le modèle vaut

Projet	CryptoBot, bot de trading piloté par le machine learning
Formation	Data Engineer, DataScientest
Dépôt	DataScientest-Studio/jul26_bde_crypto
Date	30 septembre 2026

Sommaire

1. Objectif de l'étape

2. Notions de base

- 2.1 Apprentissage supervisé
- 2.2 Découpage chronologique
- 2.3 Fuite de données
- 2.4 Extrait figé
- 2.5 Mesurer un modèle qui peut s'abstenir
- 2.6 Backtest et frais

3. Données et variables

- 3.1 Préparation
- 3.2 Des variables sans unité
- 3.3 Contexte multi-échelles
- 3.4 Des variables stables

4. Première approche : les trois barrières

- 4.1 La cible
- 4.2 Résultats
- 4.3 Trois faux résultats

5. Modèle retenu : le sens de la prochaine bougie

- 5.1 La cible et l'objectif
- 5.2 Le modèle
- 5.3 Le bouton conservateur ou agressif

6. La rentabilité

- 6.1 Le mur des frais
- 6.2 Le rejeu sur un an

7. Pistes testées

8. Suivi des expériences et export

- 8.1 MLflow
- 8.2 Le fichier du modèle

9. Conclusion

10. Annexe : fichiers

1. Objectif de l'étape

L'étape 3 consiste à entraîner des modèles de machine learning sur les données stockées, à les comparer, à optimiser le meilleur et à l'exporter pour l'API. Elle demande aussi de suivre les expériences avec MLflow et de rendre un notebook qui retrace la démarche.

Demande	Réalisation	Emplacement
Extrait figé des données	coupure au 24 août 2026, empreinte SHA-256	<code>scripts/make_extract.py</code>
Exploration des données	doublons, valeurs nulles, répartition des classes	notebook, section 2
Indicateurs techniques	26 variables sans unité, puis 35 avec le contexte multi-échelles	<code>src/features.py</code>
Plusieurs modèles comparés	6 modèles dont 2 références, puis 3 modèles sur la cible finale	<code>scripts/compare_models.py</code>
Optimisation (GridSearchCV)	validation croisée chronologique	<code>scripts/optimize_model.py</code> , <code>scripts/optimize_direction.py</code>
Stop loss et take profit	intégrés à l'étiquetage et au carnet de positions	<code>src/labeling.py</code>
Export du modèle	fichier <code>.joblib</code> avec ses variables et ses seuils	<code>models/</code>
Suivi des expériences	MLflow	<code>mlflow.db</code>
Notebook	démarche complète, exécutée	<code>notebooks/etape3_modelisation.ipynb</code>

2. Notions de base

2.1 Apprentissage supervisé

Un modèle **supervisé** apprend une relation entre des **variables explicatives** (ce que l'on sait au moment de décider) et une **cible** (ce que l'on veut prédire), à partir d'exemples dont on connaît la réponse. En **classification**, la cible est une catégorie (hausse ou baisse) ; en **régression**, c'est une valeur numérique (un prix).

Le modèle est ajusté sur un jeu d'**apprentissage**, ses réglages sont choisis sur un jeu de **validation**, et sa qualité est mesurée sur un jeu de **test** qu'il n'a jamais vu. Mesurer sur des données déjà vues revient à noter un élève sur les exercices qu'il a recopiés.

2.2 Découpage chronologique

Sur des séries temporelles, le découpage doit respecter l'ordre du temps : on apprend sur le passé et on teste sur le futur. Un découpage aléatoire mélangerait des bougies de mars dans l'apprentissage et des bougies de février dans le test ; le modèle apprendrait sur des informations postérieures à celles qu'il doit prédire.

Pour la validation croisée, nous utilisons `TimeSeriesSplit` de scikit-learn : chaque découpage entraîne sur une période et valide sur la période suivante, jamais l'inverse.

2.3 Fuite de données

Une **fuite de données** (*data leakage*) désigne une information du futur qui se glisse dans les variables. Le modèle obtient alors d'excellents scores en test, qu'il ne reproduira jamais en production, où le futur n'est pas disponible. C'est l'erreur la plus fréquente sur les séries temporelles, et la plus difficile à voir. Nous en avons trouvé une pendant le projet (section 4.3).

2.4 Extrait figé

Les données en base évoluent : de nouvelles bougies arrivent chaque jour. Entraîner deux fois sur « la base » donne deux résultats différents, sans que le code ait changé. Nous entraînons donc sur un **extrait figé** : une copie des données jusqu'à une date de coupure, enregistrée en Parquet, avec une empreinte SHA-256 de chaque fichier. L'empreinte est vérifiée avant chaque entraînement et enregistrée dans MLflow. Deux personnes qui lancent le même script obtiennent le même modèle.

2.5 Mesurer un modèle qui peut s'abstenir

L'**accuracy** est la part de réponses justes sur toutes les prédictions. Notre modèle final a le droit de ne pas se prononcer : il répond « attendre » quand il n'est pas assez sûr. Nous mesurons alors l'**accuracy sélective** : la part de réponses justes parmi les seules bougies où il a donné un avis. Un modèle qui ne se prononce que sur 2 % des bougies mais a raison 60 % du temps sur celles-ci peut être plus utile qu'un modèle qui se prononce toujours à 52 %.

Une accuracy mesurée sur quelques centaines d'ordres reste incertaine. Nous l'accompagnons de son **intervalle de confiance** à 95 %, calculé par la méthode de Wilson, adaptée aux proportions : avec 469 ordres à 59,9 %, la vraie valeur se situe entre 55,4 % et 64,3 %.

2.6 Backtest et frais

Un **backtest** rejoue les décisions du modèle sur le passé, avec un capital fictif, pour savoir ce qu'elles auraient rapporté. Il intègre les **frais de transaction** : 0,1 % à l'achat et 0,1 % à la vente sur Binance, soit 0,2 % par aller-retour. Un modèle peut avoir raison plus souvent qu'il n'a tort et perdre de l'argent, si ses gains moyens ne couvrent pas les frais.

3. Données et variables

3.1 Préparation

Tous les modèles sont entraînés sur le même extrait : 2 124 845 lignes, coupées au 24 août 2026. L'exploration n'a trouvé aucun doublon, aucune valeur manquante (hormis la période de chauffe des indicateurs, retirée) et aucune incohérence de prix, ce que les contraintes du schéma SQL garantissaient déjà.

3.2 Des variables sans unité

La bibliothèque `ta` propose une fonction qui ajoute tous ses indicateurs d'un coup. Nous ne l'avons pas utilisée : elle produit 86 colonnes, dont 231 paires presque identiques (corrélation supérieure à 0,95) et deux colonnes vides à plus de 45 %.

Nous avons retenu 26 variables réparties en cinq familles, toutes **sans unité** : des rapports et des pourcentages, jamais un prix brut. Un modèle qui apprendrait « le bitcoin vaut 63 000 dollars » serait inutilisable sur l'ETH à 3 000 dollars, et sur le bitcoin l'année suivante.

Famille	Exemples	Ce qu'elle mesure
Tendance	écart du prix à ses moyennes mobiles 10, 30 et 100, écart entre moyennes exponentielles, MACD rapporté au prix, ADX	la direction et sa force
Momentum	RSI 14, stochastique, taux de variation sur 10 bougies	la vitesse du mouvement
Volatilité	ATR rapporté au prix, écart-type des rendements sur 24 bougies, position et largeur des bandes de Bollinger, amplitude de la bougie	l'amplitude des mouvements
Volume	volume relatif, OBV normalisé, MFI, nombre de transactions relatif	l'activité par rapport à la normale
Flux acheteur	part du volume initiée par les acheteurs, écart à son habitude, taille moyenne d'un trade	qui prend l'initiative

3.3 Contexte multi-échelles

Une bougie de 15 minutes s'interprète mieux en connaissant l'état des échelles plus lentes : une hausse de 15 minutes n'a pas le même sens dans une tendance 4 h haussière ou baissière. Nous ajoutons à chaque bougie les variables principales des pas de temps 1 h et 4 h, soit 35 variables au total.

La jonction entre échelles a produit notre fuite de données. Dans la première version, une bougie 15 minutes de 12 h 15 recevait les indicateurs de la bougie 4 h ouverte à midi, qui ne se termine qu'à 16 h : quatre heures de futur. Le score était passé de 0,52 à 0,61. Corrigé en ne joignant que les bougies lentes déjà **clôturées**, il est retombé à 0,52. Un test automatisé empêche désormais cette erreur de revenir.

3.4 Des variables stables

Un test vérifie aussi que chaque variable a la même valeur quelle que soit la quantité d'historique utilisée pour la calculer. L'OBV normalisé ne le vérifiait pas : il valait 0,0089 calculé sur tout l'historique et -0,0318 sur les 500 dernières bougies, parce qu'il cumulait depuis la première bougie disponible. L'API, qui calcule sur 300 bougies, aurait donné au modèle des valeurs différentes de celles de l'entraînement. Il a été redéfini sur une fenêtre fixe, et le modèle réentraîné.

4. Première approche : les trois barrières

4.1 La cible

Prédire le prix de la bougie suivante donne un score trompeur : un modèle naïf qui répond « le prochain prix sera le prix actuel » obtient un R^2 de 0,999998, et les vrais modèles font moins bien que lui. Ce R^2 mesure surtout que le bitcoin vaut à peu près le même prix qu'une heure plus tôt.

Nous avons d'abord prédit le mouvement avec la **méthode des trois barrières** (López de Prado, 2018). Pour chaque bougie, on pose une barrière haute (take profit), une barrière basse (stop loss) et une échéance. La première touchée donne l'étiquette : acheter, vendre ou attendre. Les barrières sont placées à un multiple de la volatilité récente, notée σ : le même réglage place le take profit à +68 dollars un jour calme et à +3 576 dollars un jour agité.

4.2 Résultats

Six modèles ont été comparés par profil de trading, dont deux références (toujours la classe majoritaire, et le hasard). L'accuracy dépasse le hasard, mais elle compte aussi les bougies où le modèle répond « attendre » à raison. Mesuré seulement sur les ordres passés, le sens de la prédiction n'est juste que dans 50,6 % des cas sur le day trading : autant que le hasard.

Mesure (day trading)	Calcul	Résultat	Hasard
Accuracy	12 758 ÷ 30 000 bougies	0,425	0,333
Sens correct parmi les ordres	8 262 ÷ 16 316 ordres	0,506	0,500

Au backtest, les trois profils perdaient de l'argent, alors qu'un modèle parfait, qui connaîtrait l'avenir, aurait été rentable partout. Les barrières contenaient donc du signal exploitable ; c'est la prédiction du sens qui échouait.

4.3 Trois faux résultats

Trois résultats spectaculaires sont apparus en cherchant à améliorer le modèle. Aucun n'a résisté à la vérification.

Ce qui semblait fonctionner	Ce que la vérification a montré
Contexte multi-échelles : 0,52 vers 0,61	la fuite de données décrite en 3.3
BNB, meilleure paire, $p < 0,0001$	une seule période favorable ; sur les précédentes, BNB était sous le hasard
Classement des modèles par paire	corrélation de -0,24 entre deux périodes : aucun pouvoir prédictif

Une p-value calculée sur une seule période ne prouve rien ; seule la réplication sur d'autres périodes le fait.

5. Modèle retenu : le sens de la prochaine bougie

5.1 La cible et l'objectif

Après la première approche, nous avons retenu avec notre mentor une cible plus simple et un objectif mesurable : **prédire le sens de la prochaine bougie** (hausse ou baisse) sur le profil day trading (15 min, 1 h et 4 h), avec au moins 60 % de bonnes réponses sur les bougies où le modèle se prononce. Le modèle peut s'abstenir la plupart du temps : s'abstenir 80 % du temps est acceptable s'il a raison quand il parle.

5.2 Le modèle

Élément	Choix	Justification
Algorithme	forêt aléatoire, 400 arbres, au moins 100 exemples par feuille	meilleur score en validation parmi forêt, gradient boosting et régression logistique
Variables	35, avec le contexte 1 h et 4 h	les autres pistes n'ont rien apporté de régulier (section 6)
Calibration	isotonique, sur une période que le modèle n'a pas vue	seule amélioration confirmée : +1,5 à +1,7 point d'accuracy sélective
Découpage	72 % apprentissage, 8 % calibration, 20 % réglage des seuils	chronologique
Mesure finale	14 831 bougies postérieures à l'extrait, jamais vues	aucune décision n'a été prise sur elles

Une **calibration** corrige les probabilités d'un modèle pour qu'elles correspondent aux fréquences observées : quand le modèle calibré annonce 58 %, la hausse se produit environ 58 fois sur 100. Une forêt aléatoire non calibrée est trop prudente sur les valeurs extrêmes ; or ce sont précisément celles sur lesquelles un modèle sélectif décide.

5.3 Le bouton conservateur ou agressif

Le modèle donne une probabilité de hausse. Le **style** fixe à partir de quelle probabilité le bot agit : au-dessus d'un seuil il achète, en dessous d'un autre il vend, entre les deux il attend.

Chaque style a **deux seuils**, un par côté. Les probabilités du modèle ne sont pas symétriques : elles descendent bas du côté de la baisse mais ne dépassent jamais 0,564 du côté de la hausse. Avec un seuil unique mesuré depuis 0,5, le style conservateur se trouvait au-dessus de ce maximum et ne pouvait jamais acheter. Chaque seuil est réglé pour que chaque côté déclenche la même part d'ordres.

Style	Achète si $p \geq$	Vend si $p \leq$	Ordres par jour	Accuracy	Intervalle à 95 %
Agressif	0,5346	0,4315	25	0,597	0,557 à 0,636
Conservateur	0,5415	0,3921	20	0,599	0,554 à 0,643

Mesures sur les 14 831 bougies jamais vues (24 août au 16 septembre 2026). L'objectif de 60 % est atteint en moyenne, et la borne basse de l'intervalle reste nettement au-dessus du hasard.

6. La rentabilité

6.1 Le mur des frais

Avoir raison 60 % du temps ne suffit pas. Sur une bougie de 15 minutes, le prix bouge en moyenne de 0,227 %. Un aller-retour coûte 0,2 %. Pour qu'un ordre gagne en moyenne, il faut que la précision p vérifie :

$$p \times \text{mouvement} - (1 - p) \times \text{mouvement} - \text{frais} > 0$$

$$p > (\text{mouvement} + \text{frais}) / (2 \times \text{mouvement})$$

Pas de temps	Mouvement moyen	Précision nécessaire	Précision obtenue
15 min	0,227 %	94 %	60 %
4 h	0,67 %	65 %	53 à 57 %
1 jour	1,77 %	56 %	48 à 58 %, instable

Au backtest sur les bougies jamais vues, le style agressif perd 5,0 % et le conservateur 3,6 %, frais compris. Sans frais, ils gagneraient 0,8 et 1,0 %.

6.2 Le jeu sur un an

Le carnet de positions de l'étape 4 a été rejoué sur un an de bougies (13 367 positions, avec take profit et stop loss). En 15 minutes, 50,2 % des trades agressifs sont gagnants et le capital passe de 10 000 à 7 547 euros. En 4 heures, 53,4 % de gagnants et le capital termine à 10 154 euros. Le taux de réussite est presque le même ; c'est le rapport entre le mouvement et les frais qui fait la différence.

Une mesure complémentaire confirme le diagnostic. Le **ratio d'excursion** compare, pour chaque trade, l'amplitude du mouvement en sa faveur (MFE) à l'amplitude du mouvement contre lui (MAE). Il vaut 0,92 en médiane : le prix va en moyenne un peu plus loin contre la position que pour elle. L'avantage du modèle sur le sens d'une bougie ne se prolonge pas au-delà.

7. Pistes testées

Chaque piste a été évaluée avec la même règle, fixée avant les essais : elle est retenue si elle améliore l'accuracy sélective à 2 % et à 5 % d'activité sur la période de validation.

Piste	Résultat	Décision
Calibration des probabilités	+1,5 point à 2 %, +1,7 point à 5 %	retenue
Calendrier (heure, jour de la semaine)	gain à 5 %, perte à 2 %	écartée
État du bitcoin pour les autres paires	gain faible à 5 %, perte à 2 %	écartée
Valeurs des bougies précédentes	aucun gain régulier	écartée
Carnet d'ordres des contrats à terme	aucun gain régulier sur deux ans d'historique	écartée
Horizons plus longs (4 h, 1 jour)	résultats instables d'une période à l'autre, toujours sous la précision nécessaire pour couvrir les frais	écartée
Viser les grands mouvements	même en sachant à l'avance quelles bougies bougeront le plus, le sens n'y est pas mieux prédit (48 % sur les 10 % plus grands)	écartée
Reconnaître les grosses bougies	précision de 8 à 10 % pour une fréquence de base de 2 % : réel mais inexploitable	écartée

La conclusion commune est conforme à la littérature : la volatilité d'un marché se prédit assez bien, sa direction beaucoup moins.

8. Suivi des expériences et export

8.1 MLflow

MLflow enregistre chaque entraînement : paramètres, métriques (accuracy, intervalles, backtest), empreinte de l'extrait figé et modèle lui-même. On peut dire, pour n'importe quel modèle, sur quelles données et avec quels réglages il a été produit. À l'étape 5, MLflow devient un serveur partagé et tient le **registre des modèles** : chaque nouveau modèle y devient une version, et l'alias `champion` désigne celui qui est en service.

8.2 Le fichier du modèle

Le modèle est exporté dans `models/direction_day_trading.joblib`. Le fichier contient tout ce qu'il faut pour prédire sans rien recalculer ailleurs :

- le modèle calibré ;
- la liste exacte des 35 variables, dans l'ordre attendu ;

- les seuils des deux styles ;
- les mesures obtenues et l'empreinte des données d'entraînement.

La version de scikit-learn est figée (1.9.0) dans l'API comme dans l'entraînement : un modèle chargé avec une autre version peut produire des prédictions différentes sans lever d'erreur.

9. Conclusion

La chaîne de machine learning est complète et vérifiable : extrait figé, découpage chronologique, références systématiques, mesure sur des données jamais vues, intervalles de confiance et backtest avec frais. Le modèle atteint l'objectif fixé, 60 % de bonnes réponses sur le sens de la bougie suivante quand il se prononce. Il n'est pas rentable pour autant : sur des bougies de 15 minutes, les frais absorbent l'avantage. La piste la plus sérieuse pour la suite est de changer de cible, en ne prédisant que les mouvements assez grands pour couvrir les frais.

10. Annexe : fichiers

Fichier	Rôle
<code>scripts/make_extract.py</code>	extrait figé et vérification des empreintes
<code>src/features.py</code>	calcul des 26 variables et du contexte multi-échelles
<code>src/labeling.py</code>	étiquetage par les trois barrières
<code>scripts/compare_models.py</code> , <code>scripts/optimize_model.py</code>	première approche
<code>scripts/pistes_amelioration.py</code>	évaluation des pistes
<code>scripts/train_direction_final.py</code>	modèle final, calibration, seuils, MLflow
<code>notebooks/etape3_modelisation.ipynb</code>	démarche complète, exécutée
<code>docs/modele_final_direction.json</code>	mesures du modèle final